Shield-Chat Security Map

Where every piece of security code lives in the project, and what each piece does.

Shield-Chat is a role-based secure messaging portal. Security is layered: secrets validation at boot, a hardened HTTP middleware pipeline, encrypted data at rest, JWT + session-tracked auth, RBAC, rate limiting, input sanitisation, audit logging, and a locked-down container image. This document maps each concern to the exact file(s) that implement it.

1. At-a-glance map

Security concern	Primary file(s)
Environment / secrets validation	`backend/src/config/environment.ts`
Security constants (lockout, policy, rate limits)	`backend/src/config/constants.ts`
Central security config (CSP, CORS, cookies, JWT)	`backend/src/config/security.ts`
RBAC role + permission map	`backend/src/config/roles.ts`
HTTP security headers (Helmet)	`backend/src/middleware/securityHeaders.ts`
Per-request CSP nonce	`backend/src/middleware/cspNonce.ts`
Strip Server / X-Powered-By	`backend/src/middleware/serverHeader.ts`
CSRF (double-submit cookie)	`backend/src/middleware/csrfProtection.ts`
CORS allow-list + tunnel patterns	`backend/src/middleware/corsMiddleware.ts + config/security.ts`
JWT verification on HTTP routes	`backend/src/middleware/authMiddleware.ts`
Role-based access control on routes	`backend/src/middleware/roleMiddleware.ts`
Rate limiting (login, API, messages, reset)	`backend/src/middleware/rateLimiter.ts`
Input validation (express-validator)	`backend/src/middleware/validationMiddleware.ts`
Generic error handling (no info leak)	`backend/src/middleware/errorHandler.ts`
AES-256-GCM encrypt/decrypt	`backend/src/utils/encryption.ts`
bcrypt hashing + password policy	`backend/src/utils/passwordUtils.ts`
JWT sign/verify (access + refresh)	`backend/src/utils/tokenUtils.ts`
XSS / NoSQL input sanitisation	`backend/src/utils/sanitizers.ts`
Login, register, lockout, password reset	`backend/src/services/authService.ts`
Token rotation, session revocation	`backend/src/services/tokenService.ts`
Audit log writes (immutable)	`backend/src/services/auditService.ts`
Email + message encryption helpers	`backend/src/services/encryptionService.ts`
User model: lockout, password history	`backend/src/models/User.ts`
Socket.io JWT auth	`backend/src/websocket/socketAuth.ts`
Middleware pipeline (ordering)	`backend/src/app.ts`
CSRF auto-attach (axios xsrf options)	`frontend/src/services/api.ts`
CSP nonce placeholder in SPA shell	`frontend/public/index.html`

Non-root container, no source maps	`Dockerfile`
Build-context exclusions (.env, secrets)	`.dockerignore`

2. Configuration & secrets

`backend/src/config/environment.ts`

What it does. Centralises all environment-variable reading and validates required values at boot. The app refuses to start if any secret is missing or weak.

- Throws if **JWT_ACCESS_SECRET** or **JWT_REFRESH_SECRET** is shorter than 32 characters.
- Throws if **ENCRYPTION_KEY** is not exactly 64 hex chars (32 bytes — required for AES-256).
- Loads **MONGODB_URI**, JWT expiry, bcrypt rounds, CORS origin list, rate-limit windows, log level.
- Exports a frozen env object so no other module can mutate the configuration.

`backend/src/config/constants.ts`

What it does. All security-relevant numbers in one place — no magic numbers scattered through the code.

- **RATE_LIMITS**: 5 logins / 15 min, 100 API / min, 10 messages / min, 3 password resets / hour, etc.
- **LOCKOUT_POLICY**: 5 failed attempts → 30-minute account lock.
- **PASSWORD_POLICY**: min 8 chars, max 72, requires upper + lower + digit + special, history depth 5.
- **TOKEN_EXPIRY**: access 1 h, refresh 7 d, password-reset 1 h.
- **AUDIT_ACTIONS** + **AUDIT_SEVERITY** enums used by every audit-log write.
- **HTTP_STATUS**, **COOKIE_NAMES**, **ROLES**, **ROOM_ACCESS** constants.

`backend/src/config/security.ts`

What it does. Central object describing the application's security posture: bcrypt rounds, JWT options, cookie flags, CORS rules, and CSP directives.

- **BCRYPT_ROUNDS** — work factor for password hashing (12 in production).
- **JWT_ACCESS_OPTIONS** / **JWT_REFRESH_OPTIONS** — HS256, issuer *shield-chat*, audience *shield-chat-client*, with explicit expiries.
- **ACCESS_TOKEN_COOKIE_OPTIONS** / **REFRESH_TOKEN_COOKIE_OPTIONS** — httpOnly, Secure (in prod), SameSite=Strict, refresh cookie scoped to /api/auth/refresh.
- **CORS_OPTIONS** — wildcard-free origin function: explicit allow-list + regex patterns for *.trycloudflare.com, *.ngrok-free.app, *.serveo.net, etc. Private LAN ranges (192.168.*, 10.*, 172.16-31.*, 127.*) only accepted in non-production.
- **CSP_DIRECTIVES** — nonce-based script-src / style-src-elem, script-src-attr 'none', frame-ancestors 'none', base-uri 'self', form-action 'self', object-src 'none'; the nonce is read at request time from res.locals.cspNonce.

`backend/src/config/roles.ts`

What it does. RBAC source of truth: enumerates every permission and maps each role to the permissions it grants.

- **ROLE_HIERARCHY**: employee=1 < manager=2 < admin=3 — used by isAtLeastRole() for "at least manager" style checks.
- **PERMISSIONS** object: USER_VIEW_ALL, USER_ASSIGN_ROLE, MESSAGE_DELETE_ANY, AUDIT_VIEW, AUDIT_EXPORT, etc.
- **ROLE_PERMISSIONS**: explicit per-role list — default-deny (anything not listed is denied).
- hasPermission(role, perm) and getRolesWithPermission(perm) helpers.

3. The HTTP middleware pipeline

Order matters. backend/src/app.ts wires middleware in a security-first sequence so every response — including errors — gets headers set, server fingerprints stripped, and CSRF / auth checks applied before any route logic runs.

Step	Why it's there
`1. app.disable('x-powered-by')`	Suppresses Express's default fingerprint.
`2. stripServerHeaders`	Removes Server / X-Powered-By again on every response, plus on writeHead.
`3. cspNonce`	Mints a fresh base64 nonce into res.locals.cspNonce — must run BEFORE helmet.
`4. securityHeaders (helmet)`	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, COOP, CORP, etc.
`5. corsMiddleware`	Origin allow-list (explicit + tunnel regex). Handles OPTIONS preflight.
`6. cookieParser`	Parses Cookie header so JWT and XSRF cookies are readable.
`7. express.json + urlencoded`	10 KB body cap to defeat oversized-payload DoS.
`8. mongoSanitize`	Strips $ and . from req.body / query / params — blocks NoSQL injection.
`9. trust proxy = 1`	Honour X-Forwarded-For from the first proxy hop only — accurate IPs for rate limiting.
`10. issueCsrfToken + verifyCsrfToken`	Double-submit cookie CSRF; bearer-auth requests are exempt.
`11. /api routes`	All API routes mounted here.
`12. SPA fallback`	Serves index.html with __CSP_NONCE__ replaced per request.
`13. notFoundHandler`	Generic 404 — never echoes the path.
`14. globalErrorHandler`	Last middleware. Logs internally; returns generic messages to clients (no stack traces).

Pipeline files

`backend/src/middleware/securityHeaders.ts`

What it does. Helmet configuration — emits the full set of security response headers.

- **Content-Security-Policy** with per-request nonces; useDefaults: false so we don't emit upgrade-insecure-requests (would break plain-http LAN access).
- **HSTS**: max-age 1 year, includeSubDomains. Browsers ignore HSTS over plain http (RFC 6797 §8.1) so this is safe everywhere.
- **frameguard: deny** — legacy X-Frame-Options for old browsers; modern browsers honour frame-ancestors 'none' from the CSP.
- **noSniff**, **referrerPolicy**, **hidePoweredBy**, **crossOriginOpenerPolicy**, **crossOriginResourcePolicy**.
- **xssFilter: false** — modern browsers ignore this; setting 0 avoids legacy bypass vectors.

`backend/src/middleware/cspNonce.ts`

What it does. Generates 16 random bytes per request and puts them into res.locals.cspNonce. Helmet's CSP directive functions read this value when serialising the header, so every response gets a unique nonce.

- Runs BEFORE helmet so the nonce is available when the CSP header is built.
- The same nonce is also substituted into the SPA's index.html placeholder so the inline webpack-nonce bootstrap script is accepted.

`backend/src/middleware/serverHeader.ts`

What it does. Defense-in-depth removal of Server and X-Powered-By headers on every response.

- Calls res.removeHeader() at middleware time.
- Wraps res.writeHead so any later handler that sets these headers gets them stripped right before serialisation.
- Belt-and-braces with app.disable('x-powered-by') in app.ts.

`backend/src/middleware/csrfProtection.ts`

What it does. Double-submit cookie CSRF protection for cookie-authenticated requests.

- **issueCsrfToken**: if no XSRF-TOKEN cookie, mints a 32-byte hex token, sets it as a non-httpOnly, SameSite=Strict cookie. JS reads it and echoes it back as the X-XSRF-TOKEN header.
- **verifyCsrfToken**: on POST/PUT/PATCH/DELETE, requires header == cookie (timing-safe compare). Mismatch → 403.
- GET/HEAD/OPTIONS and /api/health are exempt.
- Bearer-token requests are exempt — bearer auth is structurally CSRF-immune (browsers don't attach Authorization on cross-site requests).
- Non-secret token: not session-bearing, only purpose is that an attacker site can't read it (SameSite=Strict + CORS prevent that).

`backend/src/middleware/corsMiddleware.ts & CORS_OPTIONS in security.ts`

What it does. Whitelist-based CORS. Wildcards are not permitted — every origin must match an explicit string or one of a small set of tunnel hostname regexes.

- Allowed: exact origins from CORS_ORIGINS env var.
- Allowed (regex): *.trycloudflare.com, *.ngrok-free.app, *.ngrok.io, *.loca.lt, *.serveo.net, *.serveousercontent.com.
- Allowed only in non-production: private/loopback ranges so devs can test from a phone on the same Wi-Fi.
- credentials: true + explicit origins — required for cookie auth.
- Allowed headers: Content-Type, Authorization, X-CSRF-Token, X-XSRF-TOKEN, X-Requested-With.

`backend/src/middleware/authMiddleware.ts`

What it does. JWT verification on protected routes.

- **verifyToken**: reads token from Authorization: Bearer header first, then the access_token httpOnly cookie. Verifies signature + issuer + audience + algorithm + expiry.
- Re-checks the user in the DB on every request — catches deleted / deactivated accounts even if the token is still cryptographically valid.
- Uses the *DB* role (not the role baked into the token) so role changes take effect immediately.
- Returns 401 with a generic message on any failure; logs the failure name at debug level.
- **optionalAuth**: same flow but never blocks — used for endpoints that work both authed and unauthed.

`backend/src/middleware/roleMiddleware.ts`

What it does. RBAC enforcement factory.

- requireRole('admin') / requireRole('manager','admin') — explicit allow-list.
- Default-deny: any role not listed gets 403.
- Unauthorised attempts are written to the audit log (action UNAUTHORIZED_ACCESS, severity warning).
- Convenience wrappers requireAdmin, requireManager, requireAnyRole.

`backend/src/middleware/rateLimiter.ts`

What it does. express-rate-limit instances tuned per endpoint category. Every breach is audit-logged.

- **loginRateLimiter** — 5 / 15 min, keyed by IP. Brute-force defence.
- **apiRateLimiter** — 100 / min, keyed by userId (falls back to IP).
- **unauthRateLimiter** — 50 / min, keyed by IP. Anti-scraping.
- **messageRateLimiter** — 10 / min per user. Anti-spam in chat rooms.
- **readRateLimiter** — 500 / min per user. Polling guardrail.
- **passwordResetRateLimiter** — 3 / hour, keyed by IP. Stops email enumeration / mailer abuse.

`backend/src/middleware/validationMiddleware.ts`

What it does. Runs express-validator chains and returns structured 400 responses with field-level messages so the client can show form feedback.

- Validations run in parallel for speed.
- Field error messages are specific (e.g. "Username is required") — login errors stay generic to prevent enumeration; validation errors do not.

`backend/src/middleware/errorHandler.ts`

What it does. Last middleware in the chain. Catches all next(err) calls and returns generic messages so internals never leak.

- Always logs err.message, err.name, err.stack, req.path, req.user.userId to winston for ops.
- Never sends stack traces, error names, or driver-specific text in the response.
- Maps Mongoose ValidationError, CastError, duplicate-key (11000), JWT errors, and PayloadTooLarge to safe canned messages.
- Falls through to a generic 500 with SERVER_ERRORS.INTERNAL message.

4. Crypto, hashing, tokens

`backend/src/utils/encryption.ts`

What it does. Low-level AES-256-GCM authenticated encryption used for emails and chat messages at rest.

- encrypt(plaintext): random 16-byte IV per call, returns iv:authTag:ciphertext base64 string.
- decrypt(text): reconstructs IV + auth tag + ciphertext, sets the auth tag, decrypts. Tampering → throws (auth tag mismatch).
- Generic error message thrown on failure — never leaks driver text.
- generateSecureToken(bytes) for password-reset tokens; hashToken() SHA-256 for DB storage.
- isEncrypted() guards against double-encryption by sniffing the format.

`backend/src/utils/passwordUtils.ts`

What it does. All password handling: hashing, comparison, strength checks, and reuse history.

- hashPassword — bcrypt at **12 rounds**; rejects empty input or input over 72 bytes (bcrypt silently truncates beyond that).
- comparePassword — bcrypt's timing-safe compare; returns false on error rather than throwing (no oracle).
- validatePasswordStrength — enforces PASSWORD_POLICY: length, upper/lower/digit/special, no all-same-character passwords; returns the full list of violated rules.
- generateResetToken — 32 random bytes; returns the raw token (for the email link) and its SHA-256 hash (for DB storage). Raw token is never persisted.
- checkPasswordHistory — bcrypt-compares the candidate against each of the user's last 5 hashes; blocks reuse.

`backend/src/utils/tokenUtils.ts`

What it does. JWT sign and verify for access / refresh tokens.

- generateAccessToken(userId, role) — type:'access', signed with JWT_ACCESS_SECRET, issuer/audience set.
- generateRefreshToken(userId, role) — type:'refresh', signed with the separate JWT_REFRESH_SECRET.
- verifyAccessToken / verifyRefreshToken — verify signature, expiry, issuer, audience, algorithm (HS256 only). Also assert the token-type field matches so an access token can't be replayed as a refresh token, and vice-versa.
- extractBearerToken — strict Bearer <value> parser; returns null on anything malformed.

`backend/src/utils/sanitizers.ts`

What it does. Input sanitisation — defence in depth alongside validation.

- sanitizeString — strips ALL HTML tags and attributes (whitelist = empty), removes null bytes, normalises line endings, trims.
- sanitizeObject — recursive variant for whole request bodies; depth-capped at 5 to defeat deeply-nested DoS.
- sanitizeHtml — narrow whitelist (b, i, em, strong, u, br); strips javascript: / data: URLs from any href/src.
- escapeRegex — escapes regex metacharacters to safely build MongoDB $regex queries from user input.

5. Auth, sessions, audit services

`backend/src/services/authService.ts`

What it does. Business logic for register / login / logout / forgotPassword / resetPassword / changePassword / refresh.

- **register**: validates password strength, checks username + email uniqueness, AES-encrypts the email, bcrypt-hashes the password, creates the user as 'employee' (admin assigns higher roles separately), issues a token pair, audit-logs the registration.
- **login**: case-insensitive username lookup; checks isActive, isLockedOut, then bcrypt password. Increments failed counter on bad password — at 5 failures locks for 30 min; the same generic message is returned in every failure case so attackers can't tell "wrong password" from "account locked" from "no such user".
- **forgotPassword**: *always* returns success (with a 500 ms delay) regardless of whether the email exists — prevents account enumeration.
- **resetPassword**: validates the SHA-256-hashed token, enforces password strength + history, hashes the new password, invalidates the reset token, and revokes *all* active sessions so every device has to re-login.
- **changePassword**: requires the current password (re-auth), enforces same checks, revokes all sessions.
- **refreshAuthTokens**: delegates to tokenService for rotation; audit-logs TOKEN_REFRESH on success.

`backend/src/services/tokenService.ts`

What it does. Refresh-token lifecycle with rotation and revocation.

- **createTokenPair**: issues access + refresh JWTs, stores only the SHA-256 hash of the refresh token in the Sessions collection along with IP + UA + expiry.
- **refreshTokens**: verifies JWT, looks up the session by hash, revokes it, then issues a fresh pair (rotation). Limits the window if a refresh token is stolen.
- **revokeToken**: marks a single session revoked (logout).
- **revokeAllUserTokens**: nukes every session for a user (called on password change, role change, admin force-logout).

`backend/src/services/auditService.ts`

What it does. Append-only audit log writes — every auth event, role change, admin action, and security violation lands here.

- createLog never throws — failures are logged via winston but must not break the request flow (e.g. a login still completes if audit DB is down).
- Auto-assigns severity from ACTION_SEVERITY_MAP (LOGIN_FAILURE → warning, ACCOUNT_LOCKED → critical, ROLE_CHANGE → critical, etc.).
- Captures IP via X-Forwarded-For first hop (after trust proxy = 1) and a 500-char-clamped User-Agent.
- getLogs, exportLogsAsCsv, exportLogsAsJson for the admin audit screen — paginated; CSV export quotes / escapes fields.
- No update / delete operations exist — audit entries are immutable by design.

`backend/src/services/encryptionService.ts`

What it does. Higher-level wrapper around utils/encryption.ts for the two domains that store ciphertext.

- encryptMessage / decryptMessage — chat message content; metadata (sender, room, timestamp) stays plaintext for indexing.
- encryptEmail / decryptEmail — emails are PII; storing them encrypted limits exposure on a DB compromise.
- safeDecryptMessage — returns [message unavailable] instead of crashing list views when one row is corrupt.

6. Data model security

`backend/src/models/User.ts`

What it does. Mongoose schema and instance methods for users — the source of truth for identity.

- **email** stored as the AES-256-GCM iv:authTag:cipher blob (encrypted at rest).
- **passwordHash** field has select: false — never returned by default queries; callers must explicitly opt in.
- **passwordHistory** array (also select: false): last 5 bcrypt hashes; pre-save hook trims older entries.
- **isActive** for soft-deactivation.
- **failedLoginAttempts** + **isLocked** + **lockoutExpires**: incrementFailedLogin() locks after 5; isLockedOut() auto-clears expired locks.
- toJSON transform strips passwordHash, passwordHistory, __v from any serialised response.
- findByUsername: case-insensitive exact match using regex with metacharacters escaped.

`backend/src/models/Session.ts, PasswordReset.ts, AuditLog.ts`

What it does. Supporting collections for session tracking, single-use reset tokens, and append-only audit history.

- **Session** — only stores the SHA-256 hash of the refresh token; raw token never touches the DB.
- **PasswordReset** — stores token hash + expiry; findValidToken matches hashed tokens within TTL; invalidateAllForUser burns old tokens after a successful reset.
- **AuditLog** — write-only from the application's perspective; queried via findByFilters(filters, page, limit).

7. Real-time (Socket.io)

`backend/src/websocket/socketAuth.ts`

What it does. Socket-level JWT authentication — sockets need the same identity guarantees as HTTP.

- Token sourced from handshake.auth.token first, then handshake.query.token, finally the access_token cookie in the handshake headers.
- Same verifyAccessToken call as HTTP — issuer/audience/algorithm/expiry checked.
- Re-checks the user in the DB; rejects with "Authentication failed" if the account is gone or deactivated.
- Attaches userId, role, username to socket.data for downstream event handlers.
- Generic error strings only — no detail leaks across the socket boundary.

8. Frontend security touchpoints

`frontend/public/index.html`

What it does. SPA shell. Carries no security META tags (ZAP flags meta-delivered headers as non-spec); everything authoritative comes via HTTP headers from Express.

- Single inline <script nonce="__CSP_NONCE__"> that sets window.__webpack_nonce__ so any dynamically-injected webpack chunks carry the per-request nonce.
- Server replaces the __CSP_NONCE__ placeholder on every HTML response.
- <noscript> message uses inline style="" attrs only — covered by the narrow style-src-attr 'unsafe-inline', which cannot be used to execute script.

`frontend/src/services/api.ts`

What it does. Central axios instance.

- withCredentials: true — cookies (access/refresh/XSRF) ride every request.
- xsrfCookieName: 'XSRF-TOKEN' + xsrfHeaderName: 'X-XSRF-TOKEN' — axios reads the cookie and auto-attaches the header to satisfy double-submit CSRF.
- Authorization Bearer injector pulls the access token from localStorage.
- Response interceptor: on 401, attempts a one-shot refresh-and-retry via /api/auth/refresh; queues concurrent requests during refresh; on refresh failure, clears state and redirects to /login.
- Wraps network errors (no response) in a single human-readable message — internal axios error structure never reaches the UI.

9. Container & deploy hygiene

`Dockerfile`

What it does. Multi-stage build (frontend-build → backend-build → alpine runtime).

- Frontend built with INLINE_RUNTIME_CHUNK=false so there's no inline runtime <script> (that would need a nonce CRA can't provide at build time).
- GENERATE_SOURCEMAP=false — no .map files in the production image (kills the ZAP "Source Code Disclosure" alert).
- Production stage runs npm ci --omit=dev — no devDependencies in the final image.
- Drops privileges: creates the app user/group with addgroup -S app && adduser -S -G app app, chowns /app, then USER app.
- HEALTHCHECK against /api/health via wget every 30 s.

`docker-compose.yml`

What it does. Single-image deployment plus an optional serveo SSH tunnel sidecar (--profile tunnel) for public URL exposure.

- env_file: ./backend/.env — secrets are runtime-injected, never baked into the image, so rotating keys only needs docker compose up -d.
- restart: unless-stopped; healthcheck on the API.
- Tunnel sidecar runs ssh -R 80:app:5000 serveo.net in a reconnect loop — disabled by default.

`.dockerignore`

What it does. Keeps secrets and bulk out of the build context.

- Excludes **/.env and **/.env.* (only .env.example is allowed in).
- Excludes node_modules, dist, build, coverage, logs, IDE noise, .git, docs.

10. Threat → defence cross-reference

Threat	Where it's mitigated
Credential brute-force	loginRateLimiter (5/15 min) + LOCKOUT_POLICY (5 fails = 30 min lock) + bcrypt(12) timing
Account enumeration	Generic error messages on login + forgotPassword always returns success with 500ms delay
XSS	Helmet CSP with nonce + style-src-attr scoped + sanitizeString/sanitizeObject + React's default es
CSRF	Double-submit XSRF-TOKEN cookie (SameSite=Strict) + X-XSRF-TOKEN header verified server-
Clickjacking	frame-ancestors 'none' + X-Frame-Options: DENY
Click-protocol/JS URL abuse	sanitizeHtml strips javascript: / data: protocols from href/src
MIME sniffing	X-Content-Type-Options: nosniff
Information disclosure (server)	Hide X-Powered-By + strip Server header + writeHead patch
Information disclosure (errors)	globalErrorHandler returns generic messages; full detail logged server-side only
Information disclosure (build)	GENERATE_SOURCEMAP=false; comments stripped by terser; .dockerignore excludes secrets
NoSQL injection	express-mongo-sanitize strips $ and . from inputs; escapeRegex() on user-supplied regex queries
Oversized payload DoS	express.json({ limit: '10kb' }); sanitizeObject depth-cap = 5
Password reuse	passwordHistory (last 5 bcrypt hashes); changePassword + resetPassword check before saving
Session hijack via stolen refresh token	Token rotation: each refresh revokes the old session and issues a fresh pair
Stale JWT after deactivation	Auth middleware re-fetches the user on every request; uses DB role, not token role
Email PII leakage	Email stored AES-256-GCM encrypted at rest; only decrypted in-process for owner views
Tunnel host injection	CORS allow-list uses regex hostname patterns + private-LAN allowed only in non-production
Audit log tampering	AuditLog model exposes no update/delete; failures don't block the request
MITM downgrade	HSTS max-age=31536000; includeSubDomains (browsers ignore over plain http)
Source-code disclosure	GENERATE_SOURCEMAP=false strips .map files from production

End of report. Generated from a static read of the codebase as of the most recent build. For day-to-day reference, the canonical source is the code itself — every file referenced here carries a top-of-file doc comment summarising its security role.